

Riassunto UML - Definizioni e sotto-definizioni

Ingegneria del Software - ITPS A.A. 2025/2026

Sintesi costruita dai file UML Overview, Componenti, Casi d'uso, Classi e Oggetti, Diagrammi di Sequenza. Esempi e codice sono stati esclusi; sono mantenute definizioni, sotto-definizioni e chiarimenti utili alla comprensione.

MODELLI DI SISTEMA

La modellazione dei sistemi è il processo con cui si sviluppano modelli astratti di un sistema. Ogni modello rappresenta una vista o prospettiva diversa del sistema: non pretende di descrivere tutto, ma seleziona gli aspetti più utili per capire, discutere, progettare o documentare il sistema. L'idea centrale è che non sia possibile comprendere un sistema complesso nella sua totalità senza costruire rappresentazioni semplificate.

Uso dei modelli nel ciclo di sviluppo

I modelli sono usati nell'ingegneria dei requisiti per facilitare la deduzione dettagliata dei requisiti, nella progettazione per descrivere il sistema a chi dovrà implementarlo e, dopo l'implementazione, per documentarne struttura e funzionamento. Possono riferirsi sia a sistemi esistenti, per chiarirne punti di forza e debolezza, sia a nuovi sistemi, per rendere più chiare le proposte progettuali e discutere i requisiti con gli stakeholder.

Modello come astrazione

Un modello di sistema non è una rappresentazione completa del sistema: elimina volutamente dettagli non necessari per rendere più comprensibile ciò che interessa. È quindi un'astrazione del sistema studiato, non una sua alternativa. Per lo stesso sistema possono essere prodotti più modelli, ciascuno utile a osservare un aspetto diverso.

Prospettiva esterna

La prospettiva esterna modella il contesto o l'ambiente nel quale opera il sistema. Serve a chiarire cosa è interno al sistema e cosa invece appartiene all'ambiente esterno con cui il sistema comunica o da cui riceve stimoli.

Prospettiva di interazione

La prospettiva di interazione modella le interazioni tra il sistema e il suo ambiente oppure tra i componenti interni del sistema. È utile per capire quali scambi avvengono, quali attori o componenti partecipano e quali messaggi o servizi vengono richiesti.

Prospettiva strutturale

La prospettiva strutturale modella l'organizzazione del sistema o la struttura dei dati elaborati. Mette in evidenza elementi, componenti, classi, dati e relazioni stabili tra essi.

Prospettiva comportamentale

La prospettiva comportamentale modella il comportamento dinamico del sistema e le sue risposte agli eventi. Serve a descrivere come il sistema evolve nel tempo, come reagisce a input, condizioni o messaggi e quali stati o azioni può attraversare.

UML - UNIFIED MODELING LANGUAGE

UML, Unified Modeling Language, è un linguaggio grafico per specificare, visualizzare, costruire e documentare gli artefatti di sistemi software. La sua funzione principale è fornire un linguaggio comune e standardizzato con cui rappresentare sistemi software in modo chiaro, preciso e condivisibile.

Comunicazione tra stakeholder

UML agevola la comunicazione tra i diversi stakeholder che partecipano allo sviluppo di un progetto software. La modellazione visuale tramite diagrammi standard permette ai partecipanti di comprendere lo stato di sviluppo del sistema in modo chiaro e univoco, riducendo il rischio di interpretazioni errate.

Specifica indipendente

UML consente di fornire la specifica di un modello indipendentemente da particolari linguaggi di programmazione e processi di sviluppo. Questo significa che un modello UML può essere compreso e discusso prima di scegliere o considerare l'implementazione concreta.

Supporto ai concetti di alto livello

UML supporta concetti di sviluppo di livello alto, come componenti, comunicazioni, framework e pattern. È quindi utile non solo per descrivere singole classi o funzioni, ma anche per rappresentare organizzazioni software più ampie e strutturate.

Caratteristiche di UML

UML non è un linguaggio proprietario. Dal 1997 la sua evoluzione è gestita dall'OMG, Object Management Group, ed è soggetta a procedure definite per ogni cambiamento. È formalmente definito come standard e alla sua specifica contribuiscono metodologi e importanti società del settore software.

UML e oggetti

Un modello UML ha due aspetti complementari: struttura statica e comportamento dinamico. La struttura statica descrive i tipi di oggetto necessari per modellare il sistema e le loro relazioni. Il comportamento dinamico descrive il ciclo di vita degli oggetti e il modo in cui collaborano per fornire le funzionalità richieste.

Oggetto in UML

Un oggetto rappresenta un oggetto fisico o un concetto logico esistente nel mondo reale. Nei modelli UML gli oggetti sono usati per rappresentare entità dotate di stato e comportamento, cioè elementi che possiedono dati e possono partecipare ad azioni o interazioni.

UML come sketch

UML può essere usato come sketch, cioè come insieme di diagrammi informali e incompleti creati per esplorare parti difficili dello spazio del problema o della soluzione. In questo uso l'obiettivo non è la completezza formale, ma sfruttare l'espressività visuale per ragionare

meglio.

UML come progetto

UML può essere usato come progetto, cioè con diagrammi relativamente dettagliati. In questa forma può servire al reverse engineering, per comprendere codice esistente tramite diagrammi, oppure al forward engineering, per guidare o generare parti dell'implementazione.

UML come linguaggio di programmazione

UML può essere inteso anche come linguaggio di programmazione, quando il codice viene generato automaticamente dal modello. Le slide indicano però che questo approccio è ancora in sviluppo per teoria, robustezza e usabilità degli strumenti.

STRUTTURA DELL'UML

La struttura dell'UML è composta da tre grandi parti: costituenti fondamentali, meccanismi comuni e architettura. I costituenti fondamentali comprendono entità, relazioni e diagrammi; i meccanismi comuni comprendono specifiche, ornamenti, distinzioni comuni e meccanismi di estendibilità; l'architettura riguarda la struttura organizzativa di un sistema.

Entità

Le entità sono elementi di modellazione. In UML possono essere strutturali, comportamentali, di raggruppamento o informative. Servono a rappresentare gli elementi del sistema e i concetti necessari per descriverlo.

Entità strutturali

Le entità strutturali modellano la struttura statica del sistema. Comprendono, tra le altre, classe, interfaccia, caso d'uso, componente e nodo. Sono usate per rappresentare elementi relativamente stabili dell'architettura o del dominio.

Entità comportamentali

Le entità comportamentali modellano la struttura dinamica del sistema. Comprendono interazioni, attività e macchine a stati. Sono usate per descrivere come il sistema si comporta durante l'esecuzione o al verificarsi di eventi.

Entità di raggruppamento

L'entità di raggruppamento principale è il package. Il package viene usato per raggruppare elementi semanticamente correlati, cioè elementi che appartengono alla stessa area logica o funzionale del modello.

Entità informative

L'entità informativa principale è l'annotazione. Essa può essere aggiunta al modello per fornire informazioni particolari, in modo simile a un post-it, senza modificare direttamente la struttura semantica principale del modello.

Relazioni

Le relazioni mostrano come due o più entità sono correlate. Servono a fissare legami significativi, cioè legami semantici, tra gli elementi del modello. Tra le relazioni principali compaiono dipendenza, associazione, aggregazione, composizione, contenimento, generalizzazione e realizzazione.

Dipendenza

La dipendenza indica che l'elemento sorgente dipende dall'elemento di destinazione e può essere influenzato dai cambiamenti eseguiti su di esso. È una relazione utile per mostrare che una modifica in un elemento può avere effetti su un altro.

Associazione

L'associazione descrive un insieme di collegamenti tra oggetti. In un modello strutturale indica che le istanze di una classe possono essere collegate alle istanze di un'altra classe.

Aggregazione

L'aggregazione indica che l'elemento di destinazione è parte integrante dell'elemento sorgente. È una relazione parte-tutto meno vincolante della composizione.

Composizione

La composizione è una forma forte e più vincolata di aggregazione. Indica un legame parte-tutto in cui le parti dipendono fortemente dal tutto.

Contenimento

Il contenimento indica che l'elemento sorgente contiene l'elemento destinazione. È utile per rappresentare organizzazioni gerarchiche o strutturali.

Generalizzazione

La generalizzazione indica che l'elemento sorgente è una specializzazione dell'elemento di destinazione più generale e può quindi sostituirlo. Esprime una relazione tassonomica tra un concetto generale e concetti più specifici.

Realizzazione

La realizzazione indica che l'elemento sorgente garantisce di eseguire il contratto specificato dall'elemento di destinazione. È spesso collegata al rapporto tra interfaccia e classe o componente che la implementa.

Diagrammi

I diagrammi sono viste o finestre che permettono di vedere il contenuto del modello. Un diagramma non coincide con il modello: il modello contiene il significato complessivo, mentre il diagramma ne mostra una parte o una vista.

Diagramma delle attività

Rappresenta processi e attività che costituiscono i processi. Può mostrare flussi di lavoro, attività organizzative e punti di interazione tra software ed esseri umani.

Diagramma delle classi

Rappresenta classi, caratteristiche delle classi e relazioni tra classi. In analisi può aiutare a definire un vocabolario rigoroso del dominio; in progettazione mostra classi software e loro relazioni.

Diagramma di comunicazione

Rappresenta l'interazione tra oggetti mettendo enfasi sui collegamenti. È utile quando interessa evidenziare chi comunica con chi più che la sequenza temporale dei messaggi.

Diagramma dei componenti

Rappresenta la struttura dei componenti e le loro connessioni. Serve a descrivere l'organizzazione del software in componenti, interfacce e dipendenze.

Diagramma di struttura composita

Rappresenta la scomposizione di una classe a runtime. È usato per descrivere come una classe o un componente sia internamente strutturato durante l'esecuzione.

Diagramma di deployment

Rappresenta la distribuzione degli elaborati su diversi nodi. Serve a mostrare come il software è mappato sull'hardware o su risorse computazionali fisiche.

Diagramma di interazione generale

Combina aspetti del diagramma di sequenza e del diagramma delle attività. Serve a rappresentare interazioni in una forma che include anche flussi e controllo.

Diagramma degli oggetti

Rappresenta una configurazione esemplificativa di istanze. È una fotografia di un sistema in un determinato momento, mostrando oggetti concreti e collegamenti.

Diagramma dei package

Rappresenta la struttura del sistema al momento della compilazione. È utile per descrivere l'organizzazione del software su larga scala.

Diagramma di sequenza

Rappresenta l'interazione tra oggetti con enfasi sulla sequenza temporale dei messaggi. È particolarmente utile per descrivere scenari di casi d'uso.

Diagramma di macchina a stati

Mostra come gli eventi cambiano un oggetto durante il suo ciclo di vita. È utile quando una entità può assumere stati diversi e passare da uno stato all'altro in seguito a eventi.

Diagramma di temporizzazione

Rappresenta l'interazione tra oggetti con enfasi sul tempo. È utile quando l'aspetto temporale dell'interazione è rilevante.

Diagramma dei casi d'uso

Mostra come gli utenti interagiscono con un sistema. Rappresenta attori, casi d'uso e relazioni, focalizzandosi sulle funzionalità osservabili dall'esterno.

Specifiche

I modelli UML hanno almeno due dimensioni: una grafica, costituita da diagrammi e icone, e una testuale, costituita dalle specifiche degli elementi di modellazione. Le specifiche sono la descrizione testuale della semantica di un elemento e costituiscono il vero contenuto del modello.

Ornamenti

Gli ornamenti sono informazioni aggiunte al simbolo grafico di un elemento di modellazione per rendere visibili aspetti aggiuntivi della sua semantica. Servono a precisare proprietà senza cambiare il tipo dell'elemento rappresentato.

Distinzioni comuni

Le distinzioni comuni chiariscono concetti UML ricorrenti. Tra i classificatori: attore, classe, componente, interfaccia, nodo e caso d'uso. Questi elementi descrivono ruoli, insiemi di oggetti, parti modulari, servizi, risorse computazionali e sequenze di azioni che producono valore.

Meccanismi di estendibilità

UML incorpora tre meccanismi di estendibilità: vincoli, stereotipi e valori etichettati. Essi permettono di adattare UML a contesti specifici senza alterarne la base concettuale.

Vincoli

Un vincolo definisce una condizione o una regola relativa a un elemento di modellazione che deve risultare sempre vera.

Stereotipi

Uno stereotipo definisce un nuovo elemento di modellazione UML basandosi su un elemento esistente. Serve a specializzare il significato di un costrutto standard.

Valori etichettati

I valori etichettati permettono di estendere la definizione di un elemento aggiungendo informazioni supplementari sotto forma di coppie nome-valore.

Architettura

L'architettura è la struttura organizzativa di un sistema, inclusa la sua scomposizione in parti, la loro connettività, le interazioni, i meccanismi e i principi guida che formano il progetto del sistema stesso.

Modello 4+1 viste

Il modello 4+1 viste descrive l'architettura attraverso vista logica, vista dei processi, vista di implementazione, vista di deployment e vista dei casi d'uso. La vista dei casi d'uso guida e collega le altre viste.

Vista logica

Descrive come oggetti e classi compongono il sistema e implementano il comportamento richiesto.

Vista dei processi

Modella thread e processi del sistema, catturando aspetti di design come concorrenza e sincronizzazione.

Vista di implementazione

Mostra il sistema dal punto di vista del programmatore, descrivendo componenti come librerie, framework, web service e sottosistemi, insieme alle loro connessioni.

Vista di deployment

Descrive come le componenti del sistema software sono mappate sulla struttura hardware.

Vista dei casi d'uso

Descrive le funzionalità osservabili dagli attori e fornisce il riferimento da cui derivano le altre viste architetturali.

UML: COMPONENTI

Una componente è una parte del sistema che è sostituibile, modulare, incapsula l'implementazione ed espone servizi. Anche un sottoinsieme di classi può costituire una componente se espone solo alcuni servizi; una componente può inoltre contenere altre componenti.

Indipendenza della componente

Ogni componente è indipendente e non deve interferire con le operazioni di un'altra componente. I dettagli dell'implementazione sono nascosti e l'implementazione può essere modificata senza influire sulle parti restanti del sistema.

Comunicazione tramite interfacce

Le componenti comunicano tramite interfacce ben definite. L'interfaccia è espressa in termini di operazioni parametrizzate e lo stato interno della componente non viene mostrato.

Interfaccia dei servizi forniti

Definisce i servizi che la componente fornisce ad altri componenti. Può essere considerata l'API della componente, perché stabilisce i metodi che possono essere chiamati da chi usa la componente.

Interfaccia dei servizi richiesti

Specifica i servizi che altri componenti devono fornire affinché la componente funzioni correttamente. Se tali servizi non sono disponibili, la componente non può operare correttamente. Questa interfaccia non compromette l'indipendenza della componente perché non stabilisce come quei servizi debbano essere realizzati.

Infrastrutture dei componenti

Le infrastrutture dei componenti forniscono servizi standard utilizzabili nei sistemi applicativi. Questo riduce la quantità di nuovo codice da sviluppare.

Componente white-box

Una componente white-box è una componente di cui si conoscono sia il funzionamento sia l'implementazione. La sua implementazione è aperta a modifiche.

Componente black-box

Una componente black-box è una componente di cui si conosce il funzionamento, ma non l'implementazione. L'implementazione non è aperta a modifiche.

Componente grey-box

Una componente grey-box è una componente di cui si conosce il funzionamento e la cui implementazione è aperta a modifiche, anche se non è necessario o non si vuole conoscerla.

per usarla.

Diagramma dei componenti

Il diagramma dei componenti è un grafico che mostra le componenti e le relazioni di dipendenza tra esse. Non mostra le istanze delle componenti, ma può mostrare il contenuto di una componente, come classi o sottocomponenti.

Componenti black-box, grey-box e white-box nel diagramma

Nel diagramma una componente black-box o grey-box può essere rappresentata senza mostrarne il contenuto; una componente white-box può invece mostrare le classi o le componenti interne che la costituiscono.

Operazioni esposte dall'interfaccia

Per specificare quali operazioni vengono esposte da una interfaccia si usano annotazioni. Per le componenti white-box è opportuno indicare anche le classi che forniscono tali operazioni.

Mapping tra classi e componenti

Nel diagramma delle componenti si possono mostrare le classi contenute in ciascuna componente white-box. Nel diagramma delle classi si può invece mostrare la componente a cui ciascuna classe appartiene. Questo collega la vista strutturale delle classi con la vista architetturale dei componenti.

Notazioni equivalenti

Alcune relazioni o componenti possono essere rappresentate con notazioni grafiche equivalenti. Nel caso della dipendenza d'uso, l'associazione di tipo usa o produce può essere espressa con forme grafiche diverse ma semanticamente equivalenti.

UML: CASI D'USO

La vista dei casi d'uso cattura il comportamento del sistema così come appare dall'esterno e partiziona le funzionalità in transazioni significative per gli attori. È quindi una vista orientata alle funzionalità osservabili e al valore fornito agli utenti o agli altri sistemi esterni.

Scenario

Uno scenario è una sequenza di azioni che caratterizza una particolare interazione tra uno o più attori e un sistema. Tale sequenza deve produrre un risultato osservabile e significativo per gli attori coinvolti.

Caso d'uso

Un caso d'uso è un insieme di scenari che hanno in comune lo stesso obiettivo. Ogni scenario appartiene a uno e un solo caso d'uso. Il caso d'uso descrive il comportamento offerto dal sistema per produrre un risultato, senza riferimento alla struttura interna del sistema.

Proprietà dei casi d'uso

I casi d'uso sono descritti in linguaggio naturale. Devono essere dichiarazioni precise di funzionalità del sistema e servono a modellare il comportamento esterno, cioè ciò che il sistema fa per gli attori, non come internamente lo realizza.

Scenario base

Lo scenario base è lo scenario che termina con esito positivo e ha uno svolgimento lineare. Rappresenta il percorso principale, ordinario, con cui il caso d'uso raggiunge il suo obiettivo.

Scenario alternativo

Lo scenario alternativo descrive un percorso diverso da quello principale. Può terminare con successo nonostante complicazioni oppure può rappresentare un fallimento o un errore.

Utilità dei casi d'uso

I casi d'uso nascono per fornire descrizioni di utilizzo del sistema facilmente comprensibili e validabili anche da persone prive di competenze informatiche. Sono utili perché rendono discutibili e controllabili i requisiti funzionali.

Diagrammi dei casi d'uso

Un diagramma dei casi d'uso mostra attori e casi d'uso insieme alle relazioni tra questi elementi. I casi d'uso rappresentano le funzionalità del sistema manifestate agli attori esterni che interagiscono con esso.

Rigorosità dei diagrammi

I diagrammi dei casi d'uso possono essere usati con diversi livelli di rigorosità. Nelle prime fasi servono soprattutto a comprendere le necessità dell'utente; nelle fasi successive, quando entrano nel processo di sviluppo, devono essere più rigorosi e precisi.

Funzioni dei diagrammi dei casi d'uso

I diagrammi definiscono chiaramente i confini del sistema, evidenziano gli attori umani e non umani, mostrano il comportamento del sistema e aiutano a rilevare incoerenze e lacune nell'analisi dei requisiti. Se ben strutturati, possono anche costituire documentazione e base per scenari di test.

Modellazione dei casi d'uso

La modellazione dei casi d'uso prevede tre attività principali: individuare un potenziale confine del sistema, individuare gli attori e individuare i casi d'uso con le principali sequenze di eventi alternative.

Confine del sistema (Subject)

Il confine del sistema definisce cosa fa parte del sistema e cosa non ne fa parte. È determinato da chi o cosa usa il sistema, cioè gli attori, e dai benefici o funzioni che il sistema offre, cioè i casi d'uso. Graficamente è rappresentato da un rettangolo: gli attori sono fuori dal rettangolo, i casi d'uso dentro.

Attore

Un attore è un ruolo o un insieme di ruoli che qualcuno o qualcosa, esterno al sistema, svolge interagendo con il sistema. È sempre esterno, interagisce direttamente con il sistema e rappresenta una prospettiva d'uso generica, che può essere svolta da persone, software o hardware.

Attore vs utente

L'utente è il soggetto o oggetto fisico che può utilizzare il sistema. L'attore è invece il ruolo che descrive una prospettiva d'uso del sistema software. Uno stesso utente può assumere più ruoli di attore e uno stesso ruolo può essere svolto da più utenti.

Generalità dei casi d'uso

I casi d'uso devono essere specificati in forma testuale e possono essere accompagnati da rappresentazioni grafiche. Modellano i requisiti funzionali del sistema per come esso appare dall'esterno, cioè secondo una vista black-box. Possono essere attivati da un attore oppure dal sistema stesso.

Requisiti coperti dai casi d'uso

I casi d'uso descrivono soprattutto requisiti funzionali. Non sono di norma esaustivi rispetto a tutti i requisiti, perché requisiti non funzionali o condizioni di sviluppo possono richiedere altri strumenti descrittivi.

Template del caso d'uso

La descrizione testuale di un caso d'uso può includere nome, identificatore, breve descrizione, attori primari, attori secondari, precondizioni, sequenza principale degli eventi, postcondizioni e sequenze alternative.

Nome del caso d'uso

Il nome deve esprimere il processo o la funzione di business modellata. Poiché rappresenta un comportamento, deve essere formulato come verbo o predicato verbale.

Identificatore

L'identificatore è un numero univoco che non cambia nel tempo, anche se il nome del caso d'uso dovesse cambiare per esigenze semantiche.

Breve descrizione

La breve descrizione riassume l'obiettivo del caso d'uso e il risultato atteso, chiarendo la responsabilità funzionale del caso d'uso.

Attori primari

Gli attori primari sono gli attori che avviano effettivamente il caso d'uso. Possono essere più di uno.

Attori secondari

Gli attori secondari sono gli attori che interagiscono con il caso d'uso dopo che è stato avviato. Possono intervenire anche in momenti diversi.

Precondizioni

Le precondizioni specificano ciò che deve essere vero prima che il caso d'uso possa iniziare. Se una precondizione non è soddisfatta, il caso d'uso non è eseguibile.

Postcondizioni

Le postcondizioni specificano ciò che deve essere vero dopo che il caso d'uso è stato eseguito, cioè lo stato del sistema al termine dell'esecuzione.

Sequenza principale degli eventi

La sequenza principale elenca i passi del caso d'uso. Il caso d'uso inizia sempre con l'attore primario che fa qualcosa per avviarlo. Ogni passo deve essere semplice, attivo e chiaro.

Ramificazioni e ripetizioni

Le ramificazioni sono diramazioni semplici dalla sequenza principale e si esprimono con la parola chiave "se". Le ripetizioni indicano sequenze da ripetere e possono essere espresse con "per" oppure "fintantoche".

Sequenze alternative

Le sequenze alternative descrivono errori, interruzioni o ramificazioni complesse. Possono non ritornare alla sequenza principale oppure ritornare a un passo esplicitamente indicato.

Specifiche delle sequenze alternative

Ogni sequenza alternativa richiede una specifica distinta. Il nome deve fare riferimento alla sequenza principale, l'identificatore è composito, e non dovrebbe avere ulteriori sequenze alternative per evitare eccessiva complessità.

Diagramma dei casi d'uso - semantica

Semanticamente, il diagramma dei casi d'uso mostra attori e casi d'uso con le relazioni tra essi. I casi d'uso rappresentano funzionalità del sistema manifestate agli attori esterni.

Relazioni nei diagrammi dei casi d'uso

Le relazioni principali sono: generalizzazione tra attori, associazione tra attore e caso d'uso, estensione tra casi d'uso, inclusione tra casi d'uso e generalizzazione tra casi d'uso.

Generalizzazione tra attori

Una generalizzazione tra un attore padre A e un attore figlio B indica che un'istanza di B può comunicare con tutti i casi d'uso collegati ad A. Gli attori figli possiedono le caratteristiche dell'attore padre e possono aggiungerne altre.

Associazione tra attore e caso d'uso

L'associazione esprime la partecipazione di un attore a un caso d'uso. È l'unica relazione tra attori e casi d'uso e può avere direzione bidirezionale, unidirezionale oppure non specificata.

Estensione - <<extend>>

La relazione di estensione da un caso d'uso A, estendente, a un caso d'uso B, base o esteso, indica che una istanza di B può essere accresciuta, sotto determinate condizioni, dal comportamento specificato in A. L'estensione è quindi opzionale e dipende da una condizione.

Punto di estensione

Il punto di estensione è il riferimento a una posizione interna a un caso d'uso base in cui possono essere inserite sequenze di azione di un caso d'uso estendente. Ogni punto è identificato da un nome unico nel caso d'uso base e l'estensione avviene sotto il controllo di una condizione di guardia.

Inclusione - <<include>>

La relazione di inclusione indica che il comportamento di un caso d'uso A include sempre il comportamento di un caso d'uso B. Serve a estrarre parti comuni di comportamento che devono essere eseguite obbligatoriamente all'interno di più casi d'uso.

Differenza tra inclusione ed estensione

Nell'inclusione, A include B e il comportamento di B è sempre incorporato in A.

Nell'estensione, A estende B e il comportamento di B può essere esteso da A solo se si verifica una condizione di guardia.

Inclusione vs preconditione

L'esecuzione di un caso d'uso incluso implica che, al termine della sua esecuzione positiva o negativa, l'interazione riprenda dal passo immediatamente successivo a quello di inclusione. La preconditione, invece, se non è soddisfatta, impedisce l'esecuzione del caso d'uso.

Generalizzazione tra casi d'uso

La generalizzazione tra casi d'uso è una relazione tassonomica tra un caso d'uso generale e uno o più casi d'uso specifici. I casi d'uso figli rappresentano varianti più specifiche del caso d'uso genitore.

Eredità nei casi d'uso

Il caso d'uso specializzato può ereditare caratteristiche dal caso d'uso genitore, aggiungere nuove caratteristiche e ridefinire caratteristiche ereditate. Può ereditare passi, punti di estensione, associazioni, preconditioni, postcondizioni e sequenze alternative, con restrizioni diverse a seconda dell'elemento.

UML: CLASSI E OGGETTI

Una classe è il descrittore di un insieme di oggetti che condividono gli stessi attributi, operazioni, relazioni e comportamento. Può essere vista come un formato di oggetti: definisce la struttura comune di tutti gli oggetti della classe. Ogni oggetto è istanza di una sola classe.

Classi di analisi

Le classi di analisi rappresentano astrazioni ben definite nel dominio del problema e devono corrispondere a concetti concreti del mondo del business. Non sono ancora necessariamente classi implementative, ma concetti utili per descrivere il dominio e le responsabilità del sistema.

Anatomia di una classe di analisi

Una classe di analisi comprende almeno un nome obbligatorio, attributi con nomi obbligatori e operazioni. Nell'analisi, le operazioni possono essere affermazioni ad alto livello sulle responsabilità della classe; tipi restituiti e parametri si indicano solo se migliorano la comprensione del modello.

Responsabilità

Una responsabilità è un servizio che la classe offre ad altre classi. Le responsabilità devono essere coerenti con l'obiettivo della classe, in modo che la classe resti coesa e comprensibile.

Qualità di una classe di analisi

Una buona classe di analisi ha un nome che rispecchia l'intento, modella una specifica astrazione del dominio, ha un insieme ridotto e ben definito di responsabilità, possiede massima coesione interna e minima interdipendenza con altre classi.

Verifiche per una classe di analisi

Le slide suggeriscono di evitare classi isolate, troppe classi semplici, poche classi troppo complesse, classi ridotte a funzioni procedurali, classi onnipotenti e alberi di ereditarietà troppo profondi. Ogni classe dovrebbe avere un numero gestibile di responsabilità e

cooperare con altre classi per realizzare gli obiettivi del sistema.

Tecniche di miglioramento delle classi

Quando un insieme di classi viola le verifiche progettuali, si possono usare fusione, separazione o una combinazione delle due. La fusione unisce responsabilità coese sparse in più classi; la separazione divide responsabilità troppo eterogenee in classi distinte.

Individuazione delle classi

Per individuare classi si raccolgono fonti come problemi, bisogni, modello dei casi d'uso e glossario. Nei testi, nomi e predicati nominali possono indicare classi o attributi; verbi e predicati verbali possono indicare responsabilità delle classi.

Altre fonti di classi

Possono diventare classi anche oggetti fisici trattati dal sistema, moduli del dominio, interfacce con il mondo esterno e entità concettuali necessarie al funzionamento ma prive di manifestazione materiale.

Oggetto

Un oggetto è una entità discreta con confini ben definiti che incapsula stato e comportamento; è istanza di una classe. È un pacchetto coeso di dati e funzioni. L'accesso ai dati avviene chiamando funzioni rese disponibili dall'oggetto.

Collaborazione tra oggetti

Gli oggetti collaborano per eseguire le funzioni del sistema stabilendo collegamenti e scambiandosi messaggi. Quando un oggetto riceve un messaggio, cerca una operazione con segnatura corrispondente e, se esiste, la esegue.

Segnatura

La segnatura di una operazione è composta dal nome del messaggio o dell'operazione, dal numero e dai tipi dei parametri formali e dal valore restituito. Nelle slide viene precisato che contano i tipi e il numero dei parametri, non i loro nomi.

Struttura runtime di un sistema OO

La struttura runtime di un sistema orientato agli oggetti è costituita da numerosi oggetti che vengono creati, esistono per un certo tempo e poi vengono distrutti.

Classe e oggetto

Una classe definisce la struttura di tutti gli oggetti appartenenti a quella classe. Tutti gli oggetti della stessa classe hanno lo stesso insieme di attributi, operazioni e relazioni, ma possono avere valori diversi degli attributi.

Notazione della classe

La notazione UML di una classe è divisa in tre sezioni: nome della classe, lista degli attributi fondamentali e lista delle operazioni fondamentali.

Notazione dell'oggetto

Un oggetto è rappresentato con lo stesso simbolo delle classi, ma il nome è sottolineato e può essere scritto nella forma NomeOggetto:NomeClasse, per indicare che si tratta di una istanza.

Diagramma degli oggetti

Un diagramma degli oggetti è una istanza del diagramma delle classi. Mostra una istantanea del diagramma delle classi in un determinato istante di tempo.

Visibilità degli elementi di una classe

Attributi e operazioni possono avere diversi livelli di accesso rispetto alle altre classi: public, private, protected e package. La visibilità indica chi può vedere o usare un elemento della classe.

Public (+)

Un elemento public è visibile a qualunque classe: ogni elemento che può accedere alla classe può accedere anche al membro pubblico.

Private (-)

Un elemento private non è visibile ad alcuna classe esterna: solo le operazioni della classe stessa possono accedere ai membri privati.

Protected (#)

Un elemento protected è visibile alle sottoclassi della classe in cui è definito.

Package (~)

Un elemento package è visibile alle classi dello stesso package ed è privato rispetto alle classi esterne al package.

Implementazione fisica di una classe

Esiste una corrispondenza tra rappresentazione UML di una classe e implementazione in un linguaggio orientato agli oggetti. Attributi e operazioni UML diventano, rispettivamente, campi e metodi nel codice.

DIAGRAMMA DELLE CLASSI

Il diagramma delle classi è un grafico che fornisce una vista strutturale del sistema in termini di classi, attributi, operazioni e relazioni tra classi. Non fornisce informazioni temporali: descrive la struttura, non l'ordine degli eventi.

Relazioni tra classi

Le principali relazioni tra classi sono associazione, aggregazione, composizione, classe associativa, generalizzazione e dipendenza. Esse descrivono connessioni semantiche diverse tra classi e oggetti.

Relazione di associazione

L'associazione indica la presenza di una popolazione di legami relazionali tra istanze di classi. È sempre bidirezionale a livello semantico e può essere binaria, se coinvolge due classi, oppure n-aria, se coinvolge tre o più classi.

Sintassi dell'associazione

Una associazione può essere descritta con nome dell'associazione, nomi dei ruoli, molteplicità e navigabilità. Il nome dell'associazione è di solito un verbo o una frase verbale; i ruoli sono nomi o frasi nominali che descrivono il ruolo assunto dagli oggetti.

Molteplicità

La molteplicità indica quante istanze di una classe possono essere associate a una singola istanza di un'altra classe in un dato istante. Se non è indicata, non si assume alcun vincolo. Può essere espressa con intervalli come 0..1, 0..*, 1..*, 1..6, 1 oppure *.

Navigabilità

La navigabilità indica che è possibile spostarsi da un oggetto della classe origine a uno o più oggetti della classe destinazione, secondo la molteplicità. Serve a indicare in quale direzione è possibile attraversare la relazione nel modello.

Ruoli nell'associazione

I ruoli forniscono un modo per attraversare la relazione da una classe a un'altra. Possono essere usati al posto dei nomi delle associazioni e sono spesso usati nelle relazioni tra oggetti della stessa classe.

Associazioni e attributi

Una associazione tra una classe origine e una destinazione indica che gli oggetti della prima possono avere riferimenti agli oggetti della seconda. Un modo di interpretarla è che la classe sorgente possieda uno pseudo-attributo il cui tipo è la classe destinazione.

Implementazione dell'associazione

Non esiste un mapping diretto dell'associazione su un costrutto unico del linguaggio di programmazione. Può essere implementata introducendo attributi che contengono

riferimenti alle istanze associate. Riferimenti multipli possono essere rappresentati con liste, array o collezioni.

Associazione uno a molti

Una associazione uno a molti tra una classe A e una classe B descrive una connessione semantica in cui a ciascun oggetto di A possono corrispondere zero, uno o più oggetti di B, mentre a un oggetto di B corrisponde al più un oggetto di A.

Classe associativa

Una classe associativa è usata quando alcune proprietà appartengono alla relazione tra classi e non alle classi coinvolte. È spesso utilizzata nelle associazioni uno a molti o molti a molti. Contiene riferimenti agli oggetti in relazione e permette di modellare attributi propri dell'associazione.

Relazione di aggregazione

L'aggregazione è una relazione di associazione che definisce una relazione tra un tutto e le parti di cui il tutto è costituito. È una relazione parte-di. Il tutto tende a dominare e controllare la relazione, mentre la parte serve le richieste del tutto.

Semantica dell'aggregazione

L'aggregato può in alcuni casi esistere indipendentemente dalle parti, ma in altri casi no. Le parti possono esistere indipendentemente dall'aggregato. L'aggregato è incompleto se mancano alcune parti.

Proprietà dell'aggregazione

L'aggregazione gode delle proprietà transitiva e antisimmetrica. A runtime, un oggetto contenuto sopravvive all'oggetto contenitore, quindi la vita della parte non dipende rigidamente dalla vita del tutto.

Implementazione dell'aggregazione

Per realizzare l'aggregazione si definiscono attributi privati del tipo delle classi aggregate o strutture che mantengano le istanze aggregate. La dimensione degli attributi dipende dalla molteplicità. I riferimenti alle parti possono essere gestiti tramite costruttori o metodi add.

Relazione di composizione

La composizione è una relazione di aggregazione forte in cui gli oggetti contenuti non hanno vita propria, ma esistono in quanto parti della classe contenente. La classe composta è responsabile della creazione e distruzione degli oggetti componenti.

Proprietà della composizione

Nella composizione la classe composta è l'unica a poter usare le classi componenti. La molteplicità dal lato della classe composta deve essere al più uno, mentre può essere

qualsiasi dal lato delle classi componenti.

Implementazione della composizione

Per tradurre la composizione si definiscono attributi privati del tipo delle classi componenti o strutture che mantengano le istanze componenti. Le classi componenti vengono istanziate all'interno della classe composta e deallocate dalla classe composta tramite distruttore o meccanismo equivalente.

Relazione di generalizzazione

La generalizzazione è una relazione tra un elemento generico e uno specializzato. L'elemento specializzato è completamente consistente con quello generico, ma contiene più informazioni. I due elementi obbediscono al principio di sostituibilità.

Ereditarietà nella generalizzazione

Le sottoclassi ereditano attributi, operazioni, relazioni e vincoli. Possono inoltre ridefinire le operazioni. La generalizzazione implica un grado di interdipendenza molto alto ed è più forte dell'associazione.

Classe astratta

Una classe astratta rappresenta una classe base o superclasse in una realizzazione di generalizzazione, contiene una o più operazioni astratte e non può essere istanziata. Nei linguaggi come Java il modificatore abstract può dichiarare sia classi astratte sia metodi astratti.

Relazione di dipendenza

La dipendenza indica una connessione semantica tra due classi in cui una dipende dall'altra. Una modifica nell'elemento indipendente può produrre effetti sull'elemento dipendente. Le dipendenze possono esistere tra classi, tra oggetti e classi e tra package.

Dipendenza di uso

La dipendenza di uso, indicata con «usa», significa che il cliente usa il fornitore. Può accadere quando una operazione della classe A ha un parametro di classe B, restituisce un valore di classe B oppure usa un oggetto di classe B nella propria implementazione senza averlo come attributo.

UML: DIAGRAMMI DI SEQUENZA

La realizzazione dei casi d'uso mostra come le istanze delle classi di analisi interagiscono tra loro per realizzare le funzionalità del sistema. I diagrammi di sequenza sono diagrammi di interazione usati per mostrare le interazioni tra istanze specifiche che realizzano uno scenario.

Interazione

Una interazione è un insieme di messaggi scambiati da istanze di classificatori in un determinato contesto. Il contesto dell'interazione è fornito dal classificatore di contesto.

Classificatore di contesto

Nella realizzazione dei casi d'uso, il classificatore di contesto è il caso d'uso. Il comportamento del caso d'uso viene realizzato dalle istanze dei classificatori, cioè dalle linee di vita, che si scambiano messaggi.

Linea di vita

La linea di vita rappresenta un singolo partecipante a una interazione, cioè il modo in cui una istanza di uno specifico classificatore partecipa all'interazione. Può avere nome, tipo e selettore.

Nome della linea di vita

Il nome della linea di vita è opzionale e identifica la linea di vita all'interno dell'interazione.

Tipo della linea di vita

Il tipo è il nome del classificatore di cui la linea di vita è una istanza.

Selettore

Il selettore è opzionale ed è una condizione booleana usata per selezionare una singola istanza che soddisfi la condizione. Se il selettore è omesso, ci si riferisce a una istanza generica.

Messaggio

Un messaggio rappresenta un tipo specifico di comunicazione tra due linee di vita in una interazione. Può implicare una chiamata di operazione, la creazione o distruzione di una istanza, oppure l'invio di un segnale.

Messaggio di chiamata

Il messaggio di chiamata richiede alla linea di vita ricevente di invocare una operazione avente la stessa segnatura del messaggio.

Messaggio di creazione

Il messaggio di creazione indica la creazione di una istanza da parte del mittente.

Messaggio di distruzione

Il messaggio di distruzione indica la distruzione di una istanza. Nel diagramma la distruzione è indicata con un marker di distruzione.

Invio di un segnale

L'invio di un segnale è una forma di comunicazione in cui una linea di vita invia un'informazione o evento a un'altra linea di vita.

Focus di controllo o attivazione

Quando una linea di vita sta eseguendo un messaggio, si dice che ha il focus di controllo o di attivazione. Nel tempo, l'interazione sposta tale focus tra le linee di vita secondo il flusso di controllo.

Messaggio sincrono

Nel messaggio sincrono il mittente aspetta che il destinatario ritorni dall'esecuzione del messaggio. È usato quando l'esecuzione del chiamante dipende dal completamento dell'operazione chiamata.

Messaggio asincrono

Nel messaggio asincrono il mittente invia il messaggio e continua l'esecuzione senza attendere un ritorno dal destinatario.

Messaggio di ritorno

Il messaggio di ritorno rappresenta la restituzione del focus di controllo al mittente di un messaggio precedente e può riportare un valore restituito.

Messaggio trovato

Il messaggio trovato ha origine al di fuori dell'ambito dell'interazione. Lo si usa quando si vuole mostrare la ricezione di un messaggio ma non il mittente.

Messaggio perso

Il messaggio perso non raggiunge mai la sua destinazione. Può rappresentare condizioni di errore in cui i messaggi possono perdersi.

Diagramma di sequenza

Il diagramma di sequenza è un grafico che mostra una interazione tra due o più oggetti mediante una sequenza temporale di azioni. È anche la rappresentazione grafica di uno scenario di un caso d'uso.

Significato del diagramma di sequenza

Il diagramma di sequenza mostra gli oggetti, la sequenza di istanziamento, eventuali distruzioni, l'ordine di invocazione delle operazioni, eventuali iterazioni o selezioni, e le relazioni runtime tra oggetti necessarie a realizzare una singola istanza di un caso d'uso.

Scenario di base e diagramma di sequenza

Lo scenario di base di un caso d'uso è modellato da un singolo diagramma di sequenza.

Scenari alternativi e diagramma di sequenza

Gli scenari alternativi possono essere modellati con un diagramma di sequenza separato oppure all'interno del diagramma di sequenza dello scenario di base.

Elementi di un diagramma di sequenza

Gli elementi principali sono tempo, oggetti, istanziamento dell'oggetto, distruzione dell'oggetto, messaggi tra oggetti, messaggi verso sé stessi, valori restituiti e iterazioni.

Auto-delega

L'auto-delega si verifica quando un oggetto invia un messaggio a sé stesso. Nel diagramma è mostrata da una freccia che ritorna verso la lifeline dello stesso oggetto.

Valore restituito

Il valore restituito di un messaggio può essere indicato nel diagramma tramite una linea tratteggiata. Serve a mostrare il risultato prodotto da una chiamata.

Iterazione

L'iterazione indica la ripetizione di un messaggio o di una sequenza di messaggi. Nelle slide viene indicata tramite il simbolo * e una condizione di iterazione tra parentesi quadre.

Vincoli e invarianti di stato

Un messaggio ricevuto può provocare un cambiamento di stato dell'istanza, ma non tutti i messaggi comportano cambiamenti di stato. Gli invarianti di stato permettono di mostrare lo stato delle istanze sulle linee di vita.

Frammenti combinati

I frammenti combinati sono aree in cui può essere diviso un diagramma di sequenza. Ogni frammento possiede un operatore, uno o più operandi e zero o più condizioni di guardia che stabiliscono se gli operandi devono essere eseguiti.

Operatore opt

L'operatore opt, option, esegue il suo unico operando se e solo se la condizione di guardia è vera. È assimilabile a una struttura if-then.

Operatore alt

L'operatore alt, alternatives, rappresenta una scelta tra più alternative. Viene eseguito l'operando la cui condizione è vera; può essere usata la parola chiave else al posto di una espressione booleana.

Operatore loop

L'operatore loop rappresenta una ripetizione. L'operando viene eseguito ripetutamente in base alla condizione o ai limiti stabiliti.

Operatore par

L'operatore par rappresenta esecuzioni parallele. Gli operandi possono essere eseguiti in parallelo, indicando che più flussi possono procedere contemporaneamente.

Operatore break

L'operatore break interrompe l'interazione normale quando la condizione associata è vera. È usato per modellare scenari di interruzione o uscita anticipata.

Fonti usate: slide 08.0 ITPS UML Overview 2026; 08.1 ITPS UML Componenti 2026; 08.2 ITPS UML Casi Uso 2026; 08.3 ITPS UML Classi e Oggetti 2026; 08.4 ITPS UML Sequenza 2026.